

Intrinsically Typed Syntax That Works (Functional Pearl)

ANONYMOUS AUTHOR(S)

Intrinsically typed syntax promises elegant mechanizations: well-typed terms are represented by well-typed data, substitution preserves typing by construction, and type safety proofs collapse to straightforward induction. For simple calculi such as the simply-typed lambda calculus this promise is fulfilled. For realistic systems, however, the technique often becomes impractical. In dependently typed proof assistants, intrinsically typed encodings of polymorphic calculi lead to pervasive type-changing equalities: renamings and substitutions indexed by other substitutions force proofs to transport terms across equal types that are not definitionally equal. This phenomenon—informally known as transfer hell—causes even routine metatheoretic arguments to be dominated by administrative reasoning.

This functional pearl shows how to make intrinsically typed syntax work in practice for polymorphic calculi. We give extracts of a mechanization of the metatheory of System F and extend it to System $F\omega$ in Agda while avoiding transfer reasoning almost entirely. The key idea is to internalize the equational theory of the σ -calculus for substitutions as definitional equality. We rely on proved σ -calculus laws for a functional representation of substitutions and install them into Agda's rewrite mechanism, obtaining canonical behavior for substitution and renaming without explicit transports. The main technical challenge is ensuring that the resulting rewrite system remains terminating and confluent together with Agda's built-in definitional equality.

The resulting development dramatically simplifies proofs: typing preservation becomes definitional, substitution lemmas disappear, and type conversion in System $F\omega$ can be handled intrinsically. Our experience suggests a general methodology: when intrinsically typed syntax generates transports, the correct solution is often not more lemmas but better definitional equality. We distill practical guidelines for mechanizing typed calculi in dependently typed proof assistants.

ACM Reference Format:

Anonymous Author(s). 2018. Intrinsically Typed Syntax That Works (Functional Pearl). *J. ACM* 37, 4, Article 111 (August 2018), ?? pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Intrinsically typed syntax is widely recommended for mechanizing type systems in dependently typed proof assistants. By representing well-typed terms as well-typed data, typing invariants hold by construction as in a Church-style presentation, i.e., making typing preservation trivial. For small calculi this approach works beautifully. However, for realistic polymorphic calculi such as System F, mechanizations often become complicated: conceptually simple proofs become cluttered with transports across equal types and administrative equalities.

The simply-typed lambda calculus is the prime example where intrinsically typed syntax works like a charm. ?? shows typical definitions of the syntax of types, type contexts, variables, and expressions, which are indexed by contexts and types (left column). Building on these definitions it is straightforward to define a small-step operational semantics and proving type safety. This is because subject reduction holds by construction and progress follows by a simple inductive proof (right column). The style of these definitions is adapted from the textbook by ?]. We elide the definition of the substitution operator $_{\sigma}$, as we discuss substitutions in detail in ??.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

```

50 data Type : Set where
51   1 : Type
52   _⇒_ : Type → Type → Type
53
54 data Ctx : Set where
55   ∅ : Ctx
56   _▷_ : Ctx → Type → Ctx
57
58 data _≥_ : Ctx → Type → Set where
59   here : (Γ ▷ T) ≥ T
60   there : Γ ≥ T → (Γ ▷ U) ≥ T
61
62 data _⊢_ Γ : Type → Set where
63   con : Γ ⊢ 1
64   var : Γ ≥ T → Γ ⊢ T
65   lam : (Γ ▷ T) ⊢ U → Γ ⊢ (T ⇒ U)
66   app : Γ ⊢ (T ⇒ U) → Γ ⊢ T → Γ ⊢ U
67
68 data _→_ {Γ} {T} : Γ ⊢ T → Γ ⊢ T → Set where
69   β-lam : app {Γ = Γ} (lam e1) e2 → (e1 [ e2 ])
70   ξ-app : e1 → e1' → app e1 e2 → app e1' e2
71
72 data Value {Γ} : Γ ⊢ T → Set where
73   con : Value con
74   lam : (e : (Γ ▷ T) ⊢ U) → Value (lam e)
75
76 data Progress : Γ ⊢ T → Set where
77   done : Value e → Progress e
78   step : e → e' → Progress e
79
80 progress : (e : ∅ ⊢ T) → Progress e
81 progress con = done con
82 progress (lam e) = done (lam e)
83 progress (app e1 e2)
84   with progress e1
85 ... | step x = step (ξ-app x)
86 ... | done (lam e) = step β-lam

```

Fig. 1. Call-by-name simply-typed lambda calculus

The simply-typed lambda calculus therefore represents the ideal scenario: intrinsic typing elides type preservation proofs, thus making type safety proofs so concise that they can serve as textbook examples [?]. Nothing comparable to transport reasoning arises in this development as only definitional properties of substitutions are required. The remainder of this paper explains why the same approach breaks down for polymorphic calculi and how to recover some of its simplicity.

Transfer hell is the colloquial term for being forced to apply type-changing lemmas to terms that are already known to be equal. The root problem is that substitutions that are propositionally equal are not definitionally equal, so terms must repeatedly be transported across equal types. This situation is annoying because proof assistants like Agda or Coq plainly refuse to let users even state this equality.¹ This phenomenon appears when substitutions themselves carry dependent typing information. In System F and F ω , expression substitutions depend on type substitutions and type substitutions depend on renamings; composing them yields types that are equal only propositionally, forcing transports throughout proofs. One workaround is to state definitions and proofs with explicit equivalence relations, but this practice is cumbersome, inefficient, and far away from the holy grail of definitional equality, which just works.

To our knowledge, previous intrinsically typed mechanization of System F ω (e.g., [?]) follow a natural design: expression syntax is indexed by typing derivations and substitutions are indexed by substitutions at the type level. While elegant at the level of definitions, this design makes proofs brittle. Because expression substitutions depend on type substitutions, and type substitutions depend on renamings, which again depend on type renamings, routine equalities require repeated transport across equal types, leading to directly to transfer hell.

¹A situation that can be addressed using heterogeneous equality introduced in Conor McBride's thesis [?]. [?] discuss this approach in the context of an adequacy proof for a logical relation.

The message of this pearl is simple: when intrinsically typed syntax becomes complicated, the problem is often not the encoding of terms but the definition of equality on substitutions. Strengthening definitional equality can simplify metatheory more effectively than adding automation or additional lemmas.

In this paper, we examine the transfer hell arising by intrinsically-typed syntax encodings of System F and System $F\omega$. Specifically, we consider the problems arising with type-renaming-indexed renamings and type-substitution-indexed substitutions. Our solution is to strengthen definitional equality rather than to add more lemmas. We make substitution compute canonically by internalizing the equational theory of substitutions (the σ -calculus) as definitional equality using Agda's rewriting mechanism. As a consequence, extensionally equal substitutions normalize to the same form and transports disappear from routine proofs. This approach is inspired by the tactics of Autosubst [??], but internalized by rewriting [?]. We build on a standard functional representation of renamings and substitutions [?], prove the lemmas underlying the equational laws of the sigma calculus, and then inject these equations into Agda's rewrite engine, i.e., make the sigma calculus equations definitional (for type substitutions). The technical challenge is to ensure that the added equations remain terminating and confluent together with the proof assistant's built-in definitional equality.

We first exercise this program for plain System F in ??, significantly simplifying the transfer-ridden approach used by [?], and then apply the same techniques to System $F\omega$ in ?. In the latter, the main difference to previous work [?] lies in our intrinsic treatment of type conversion, which leads to significant simplifications. In particular, in our developments, transport lemmas disappear, subject reduction holds by construction, and type conversion in System $F\omega$ is handled intrinsically.

In summary, intrinsically typed syntax fails for polymorphic calculi because substitutions lack canonical equality; making them compute canonically restores the original promise of intrinsic typing.

Contributions

This functional pearl demonstrates how to mechanize polymorphic calculi using intrinsically typed syntax without pervasive transport reasoning.

- We identify the fundamental cause of “transfer hell” in intrinsically typed encodings of System F and System $F\omega$: renamings and substitutions indexed by other substitutions produce equal types that are not definitionally equal.
- We show how to eliminate these transports by installing the equational laws of the σ -calculus for substitutions as definitional equality using Agda's rewriting mechanism, taking care to make the equations compatible with the built-in definitional equality.
- We demonstrate that the resulting rewrite system is compatible with Agda's definitional equality (in particular, terminating and confluent) for the functional representation of substitutions and verify that the resulting rewrite system is terminating and confluent with respect to the proof assistant's definitional equality.
- We mechanize parts of the metatheory of System F and extend it to System $F\omega$, obtaining substantially simpler proofs and an intrinsic treatment of type conversion for the latter.
- From this experience we extract practical guidelines for designing intrinsically typed syntax in dependently typed proof assistants.

```

148 - renamings
149  $\_ \rightarrow^R \_ : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Set}$ 
150  $n_1 \rightarrow^R n_2 = \text{Fin } n_1 \rightarrow \text{Fin } n_2$ 
151
152 opaque
153   - weakening
154    $\text{wk}^R : n \rightarrow^R \text{suc } n$ 
155    $\text{wk}^R = \text{suc}$ 
156
157   - identity renaming
158    $\text{id}^R : n \rightarrow^R n$ 
159    $\text{id}^R \alpha = \alpha$ 
160
161   - extend with new variable
162    $\_ \cdot^R \_ : \text{Fin } n_2 \rightarrow n_1 \rightarrow^R n_2 \rightarrow \text{suc } n_1 \rightarrow^R n_2$ 
163    $(\alpha \cdot^R \zeta) \text{ zero} = \alpha$ 
164    $(\_ \cdot^R \zeta) (\text{suc } \alpha) = \zeta \alpha$ 
165
166   - apply renaming to variable
167    $\_ \&^R \_ : \text{Fin } n_1 \rightarrow n_1 \rightarrow^R n_2 \rightarrow \text{Fin } n_2$ 
168    $\alpha \&^R \zeta = \zeta \alpha$ 
169
170   - composition
171    $\_ \circ \_ : n_1 \rightarrow^R n_2 \rightarrow n_2 \rightarrow^R n_3 \rightarrow n_1 \rightarrow^R n_3$ 
172    $(\zeta_1 \circ \zeta_2) \alpha = \zeta_2 (\zeta_1 \alpha)$ 
173
174   - lifting
175    $\_ \uparrow^R : n_1 \rightarrow^R n_2 \rightarrow \text{suc } n_1 \rightarrow^R \text{suc } n_2$ 
176    $\_ \uparrow^R \zeta = \text{zero} \cdot^R (\zeta \circ \text{wk}^R)$ 
177
178   - apply renaming to type
179    $\_ \dots^R \_ : \text{Type } n_1 \rightarrow n_1 \rightarrow^R n_2 \rightarrow \text{Type } n_2$ 
180    $(\alpha) \dots^R \zeta = (\alpha \&^R \zeta)$ 
181    $(\forall \alpha \text{ T}) \dots^R \zeta = \forall \alpha (\text{T } \dots^R (\zeta \uparrow^R))$ 
182    $(\text{T}_1 \Rightarrow \text{T}_2) \dots^R \zeta = (\text{T}_1 \dots^R \zeta) \Rightarrow (\text{T}_2 \dots^R \zeta)$ 
183
184
185
186
187
188
189
190
191
192
193
194
195
196

```

```

- substitutions
 $\_ \rightarrow^S \_ : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Set}$ 
 $n_1 \rightarrow^S n_2 = \text{Fin } n_1 \rightarrow \text{Type } n_2$ 

opaque
  - lift renaming to substitution
   $\langle \_ \rangle : n_1 \rightarrow^R n_2 \rightarrow n_1 \rightarrow^S n_2$ 
   $\langle \zeta \rangle \alpha = (\alpha \&^R \zeta)$ 

  - extend with new type
   $\_ \cdot^S \_ : \text{Type } n_2 \rightarrow n_1 \rightarrow^S n_2 \rightarrow \text{suc } n_1 \rightarrow^S n_2$ 
   $(\text{T} \cdot^S \eta) \text{ zero} = \text{T}$ 
   $(\text{T} \cdot^S \eta) (\text{suc } \alpha) = \eta \alpha$ 

  - apply substitution to variable
   $\_ \&^S \_ : \text{Fin } n_1 \rightarrow n_1 \rightarrow^S n_2 \rightarrow \text{Type } n_2$ 
   $\alpha \&^S \eta = \eta \alpha$ 

  - lifting
   $\_ \uparrow^S : n_1 \rightarrow^S n_2 \rightarrow \text{suc } n_1 \rightarrow^S \text{suc } n_2$ 
   $\_ \uparrow^S \eta = (\text{zero} \cdot^S \lambda \alpha \rightarrow (\eta \alpha) \dots^R \text{wk}^R)$ 

  - apply substitution to type
   $\_ \dots^S \_ : \text{Type } n_1 \rightarrow n_1 \rightarrow^S n_2 \rightarrow \text{Type } n_2$ 
   $(\alpha) \dots^S \eta = \alpha \&^S \eta$ 
   $(\forall \alpha \text{ T}) \dots^S \eta = \forall \alpha (\text{T } \dots^S (\eta \uparrow^S))$ 
   $(\text{T}_1 \Rightarrow \text{T}_2) \dots^S \eta = (\text{T}_1 \dots^S \eta) \Rightarrow (\text{T}_2 \dots^S \eta)$ 

opaque
  - composition
   $\_ \circ^S \_ : n_1 \rightarrow^S n_2 \rightarrow n_2 \rightarrow^S n_3 \rightarrow n_1 \rightarrow^S n_3$ 
   $(\eta_1 \circ^S \eta_2) \alpha = (\eta_1 \alpha) \dots^S \eta_2$ 

```

Fig. 2. Renamings and substitutions on types

2 System F

2.1 Types

The syntax of System F types uses the standard de Bruijn encoding of variables by natural numbers. The index n stands for the number of type variables in scope. Variables are drawn from the type $\text{Fin } n$, i.e., the set $\{0, \dots, n-1\}$. The $\forall \alpha$ -binder introduces a new type variable, and $\Rightarrow$ is the infix function type constructor.

```

data Type (n : Nat) : Set where
  ' _ : Fin n → Type n
  ∀ α : Type (suc n) → Type n
  _ ⇒ _ : Type n → Type n → Type n

```

?? contains the first step in defining type renamings (left column) and substitutions (right column). A type renaming ζ maps variables from one scope n_1 to another n_2 . We write the type of type renamings as $n_1 \rightarrow^R n_2$. The definitions comprise (in order) weakening, the identity renaming, extending a renaming, applying a renaming to a variable, composing two renamings, lifting a renaming (to move under a binder), and applying a renaming to a type.

A this point, the astute reader may wonder about the use of `opaque` in the definitions. The function symbols defined in an `opaque` block are usable from outside, but their computational behavior is hidden. Thus, the only normal definition is $\uparrow^R$ for lifting a renaming. This selective hiding is essential for our approach because we want to establish new definitional equalities (i.e., the laws of the sigma calculus with first-class renamings) on top of the thus defined symbols. These new equalities interfere in a bad way with the standard definitional equalities, thus we hide the latter. To prove these equalities, we open the opaque definitions locally and proceed as usual.

For substitutions (right column of ??) we proceed analogously. A type substitution η maps variables from scope n_1 to types defined over scope n_2 . We write the type of type substitutions as $n_1 \rightarrow^S n_2$. The definitions (in order) comprise lifting a renaming to a substitution, extending a substitution, applying a substitution to variable, lifting a substitution (to move under a binder), applying a substitution to a type, and composing two substitutions.

?? gives the equational laws of the sigma calculus with first-class renamings [?]. All of these equalities are supported by proofs using the definitions from ??, but the proofs are not included because they are standard.

The first group gives computation rules for renamings, in particular, how to apply the different forms of renamings to a variable. The second group presents interaction laws for renamings. Most of them deal with composition.

The third group presents computation rules for substitutions starting with the application of a substitution to a variable. There is one equation for each possible form of a substitution: extended substitution, lifted renaming, and composition of substitutions. The final equation in this group characterizes lifting in terms of extension, weakening, and composition.

The fourth group contains laws about composition of substitutions, which are mostly analogous to the corresponding laws for renamings.

The fifth group presents laws that govern the composition of different kinds of term traversal. All four combinations of applying renamings and substitutions to a type are covered.

The final group contains covers laws that mediate between substitutions and renamings. The extraction of renamings from substitutions in general requires a dedicated solving strategy [?] and we have omitted some additionally required coincidence laws from the figure.

Orienting these equations from left to right results in a confluent and terminating rewrite system that we install in Agda using its rewrite pragma [?]. From now on, all equations in ?? are definitional!

As a sanity check, we show that lifting a renaming as well as the action of a renaming satisfy the functor laws. All of these lemmas hold definitionally, some correspond directly equations from the sigma calculus in ?. The latter are marked with *.²

`lift*-id` : $(\text{id}^R \{n\} \uparrow^R) \equiv \text{id}^R$

`lift*-id` = `refl`

`lift*-comp` : $(\zeta' \circ \zeta) \uparrow^R \equiv (\zeta' \uparrow^R) \circ (\zeta \uparrow^R)$

`lift*-comp` = `refl`

`ren*-id` : $T \dots^R \text{id}^R \equiv T$

²The naming of the lemmas is taken from [?] to enable direct comparisons.

```

246 - computing renamings
247 'beta-ext-zero : zero &R (α .R ζ) ≡ α
248 'beta-ext-suc : suc α &R (α' .R ζ) ≡ α &R ζ
249 'beta-id : α &R idR ≡ α
250 'beta-wk : α &R wkR ≡ suc α
251 'beta-comp : α &R (ζ1 ∘ ζ2) ≡ (α &R ζ1) &R ζ2
252 - interaction between renamings
253 'associativity : (ζ1 ∘ ζ2) ∘ ζ3 ≡ ζ1 ∘ (ζ2 ∘ ζ3)
254 'distributivity : (α .R ζ1) ∘ ζ2 ≡ (α &R ζ2) .R (ζ1 ∘ ζ2)
255 'interact : wkR ∘ (α .R ζ) ≡ ζ
256 'comp-idr : ζ ∘ idR ≡ ζ
257 'comp-idl : idR ∘ ζ ≡ ζ
258 'η-id : zero {n} .R wkR ≡ idR
259 'η-law : (zero &R ζ) .R (wkR ∘ ζ) ≡ ζ
260
261 - computing substitutions
262 beta-ext-zero : zero &S (T .S η) ≡ T
263 beta-ext-suc : suc α &S (T .S η) ≡ α &S η
264 beta-rename : α &S ⟨ ζ ⟩ ≡ (α &R ζ)
265 beta-comp : α &S (η1 ; η2) ≡ (α &S η1) ...S η2
266 beta-lift : η ↑S ≡ ('zero) .S (η ; ⟨ wkR ⟩)
267
268 - interaction between substitutions
269 associativity : (η1 ; η2) ; η3 ≡ η1 ; (η2 ; η3)
270 distributivity : (T .S η1) ; η2 ≡ (T ...S η2) .S (η1 ; η2)
271 interact : ⟨ wkR ⟩ ; (T .S η) ≡ η
272 comp-idr : η ; ⟨ idR ⟩ ≡ η
273 comp-idl : ⟨ idR ⟩ ; η ≡ η
274 η-id : ('zero {n}) .S ⟨ wkR ⟩ ≡ ⟨ idR ⟩
275 η-law : (zero &S η) .S (⟨ wkR ⟩ ; η) ≡ η
276
277 - composing renamings and substitutions
278 identityr : T ...R idR ≡ T
279 composeRS : (T ...R ζ1) ...S η2 ≡ T ...S (⟨ ζ1 ⟩ ; η2)
280 composeRR : (T ...R ζ1) ...R ζ2 ≡ T ...R (ζ1 ∘ ζ2)
281 composeSR : (T ...S η1) ...R ζ2 ≡ T ...S (η1 ; ⟨ ζ2 ⟩)
282 composeSS : (T ...S η1) ...S η2 ≡ T ...S (η1 ; η2)
283
284 - transforming substitutions to renamings
285 coincidence : T ...S ⟨ ζ ⟩ ≡ T ...R ζ
286 coincidence-comp : ⟨ ζ1 ⟩ ; ⟨ ζ2 ⟩ ≡ ⟨ ζ1 ∘ ζ2 ⟩
287
288
289
290
291
292
293
294

```

Fig. 3. Equational laws of the sigma calculus with first class renamings

```

295 weaken : Type n → Type (suc n)
296 weaken T = T ...R wkR
297
298 _[_]* : Type (suc n) → Type n → Type n
299 T [ T' ]* = T ...S (T' .S idS)
300
301 data Ctx : Nat → Set where
302   () : Ctx zero
303   _▷_ : Ctx n → Type n → Ctx n
304   _▷* : Ctx n → Ctx (suc n)
305
306 data _≡_ : Ctx n → Type n → Set where
307   zero : (Γ ▷ T) ≡ T
308   suc : Γ ≡ T → (Γ ▷ T') ≡ T'
309   suc* : Γ ≡ T → (Γ ▷*) ≡ (weaken T)
310
311 data Expr : Ctx n → Type n → Set where
312   ' : Γ ≡ T → Expr Γ T
313   λx : Expr (Γ ▷ T1) T2 → Expr Γ (T1 ⇒ T2)
314   _·_ : Expr Γ (T1 ⇒ T2) → Expr Γ T1 → Expr Γ T2
315   Λα : Expr (Γ ▷*) T → Expr Γ (∀α T)
316   _·*_ : Expr Γ (∀α T) → (T' : Type n) → Expr Γ (T [ T' ]*)

```

Fig. 4. Type contexts, variables, and expressions

```

312 ren*-id = refl - *
313
314 ren*-comp : T ...R (ζ' ◦ ζ) ≡ (T ...R ζ') ...R ζ
315 ren*-comp = refl - *

```

We also show analogous results for substitution.

```

317 lifts*-id : (idS {n} ↑S) ≡ idS
318 lifts*-id = refl
319
320 lifts*-comp : (η' ; η) ↑S ≡ (η' ↑S) ; (η ↑S)
321 lifts*-comp = refl
322
323 sub*-id : T ...S idS ≡ T
324 sub*-id = refl
325
326 sub*-var : (' α) ...S η ≡ α &S η
327 sub*-var = refl - *
328
329 sub*-comp : T ...S (η ; η') ≡ (T ...S η) ...S η'
330 sub*-comp = refl - *

```

2.2 Expressions

Working towards intrinsically-typed expressions, ?? defines type weakening and single substitution along with type contexts, expression variables, and the syntax of expressions. Type contexts and expression variables follow the work of [?]. Type contexts `Ctx` keep track of expression variables and type variables. Correspondingly, variables are implemented as typed de Bruijn indices with an extra case `suc*` to skip over type variables in the context. Skipping requires weakening of the type as it is used under one more type binding.

Expressions are indexed by a context and a type. Each expression constructor corresponds to a typing rule in System F. The rule for type application relies on single substitution for types.

The definition of a small-step operational semantics requires expression renamings and substitution to implement beta reduction for value abstractions and type abstractions. As before, we aim to define a call-by-name semantics for concision.

2.3 Expression Renaming and Substitution

For expressions, we represent renamings and substitutions directly by suitable functions, rather than axiomatic as with type renamings and substitutions. Hence, we cannot automate the simplification of expression substitutions; nevertheless, we define the primitives analogously.

2.3.1 Renaming. Expression renamings are indexed by type renamings because type variables in expression may have to be renamed at the same time as expression variables. This complication makes the types of the operations quite involved, although the actual operations are not that complicated.

$$\begin{aligned} _ \Rightarrow^R _ : n_1 \rightarrow^R n_2 \rightarrow \text{Ctx } n_1 \rightarrow \text{Ctx } n_2 \rightarrow \text{Set} \\ \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 = \forall T \rightarrow \Gamma_1 \ni T \rightarrow \Gamma_2 \ni (T \dots^R \zeta) \end{aligned}$$

Defining the identity renaming is straightforward due to the presence of rewriting for type substitutions.

$$\text{Id}^R : \text{id}^R \mid \Gamma \Rightarrow^R \Gamma$$

$$\text{Id}^R _ x = x$$

Weakening of a renaming by a value binding.

$$\text{Wk}^R : \forall T \rightarrow \text{id}^R \mid \Gamma \Rightarrow^R (\Gamma \triangleright T)$$

$$\text{Wk}^R _ _ = \text{succ}$$

Weakening of a renaming by a type binding.

$$\text{wk}^{R*} : \text{wk}^R \mid \Gamma \Rightarrow^R (\Gamma \triangleright^*)$$

$$\text{wk}^{R*} _ x = \text{succ}^* x$$

Extension of a renaming.

$$_ \cdot^R _ : \forall \zeta \rightarrow \Gamma_2 \ni (T \dots^R \zeta) \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow \zeta \mid (\Gamma_1 \triangleright T) \Rightarrow^R \Gamma_2$$

$$(_ \mid x \cdot^R \rho) _ \text{zero} = x$$

$$(_ \mid _ \cdot^R \rho) _ (\text{succ } x) = \rho _ x$$

Lifting of a renaming over a value binding.

$$_ \uparrow^R _ : \forall \zeta \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow \forall T \rightarrow \zeta \mid (\Gamma_1 \triangleright T) \Rightarrow^R (\Gamma_2 \triangleright (T \dots^R \zeta))$$

$$(\zeta \mid \rho \uparrow^R _) = \zeta \mid \text{zero} \cdot^R (\zeta, \text{id}^R \mid \rho \odot (\text{Wk}^R _))$$

Lifting of a renaming over a type binding.

$$_ \uparrow^{R*} : \forall \zeta \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow (\zeta \uparrow^R) \mid (\Gamma_1 \triangleright^*) \Rightarrow^R (\Gamma_2 \triangleright^*)$$

$$(_ \mid \rho \uparrow^{R*}) _ (\text{succ}^* x) = \text{succ}^* (\rho _ x)$$

Inductive definition of a renaming on expressions. The last case of the definition requires rewriting to type check.

$$_ \dots^R _ : (\zeta : n_1 \rightarrow^R n_2) \rightarrow \text{Expr } \Gamma_1 \ T \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow \text{Expr } \Gamma_2 \ (T \dots^R \zeta)$$

$$_ \mid (\text{' } x) \dots^R \rho = \text{' } (\rho _ x)$$

$$_ \mid (\lambda x \ e) \dots^R \rho = \lambda x \ (_ \mid e \dots^R (\rho \uparrow^R _))$$

$$_ \mid (\Lambda \alpha \ e) \dots^R \rho = \Lambda \alpha \ (_ \mid e \dots^R (\uparrow^{R*} \rho))$$

$$_ \mid (e_1 \cdot e_2) \dots^R \rho = (_ \mid e_1 \dots^R \rho) \cdot (_ \mid e_2 \dots^R \rho)$$

$$\zeta \mid (e \cdot^* T') \dots^R \rho = (\zeta \mid e \dots^R \rho) \cdot^* (T' \dots^R \zeta)$$

2.3.2 Substitution. Expression substitutions are indexed by type substitutions.

$$_ \Rightarrow^S _ : n_1 \rightarrow^S n_2 \rightarrow \text{Ctx } n_1 \rightarrow \text{Ctx } n_2 \rightarrow \text{Set}$$

$$\eta \mid \Gamma_1 \Rightarrow^S \Gamma_2 = \forall T \rightarrow \Gamma_1 \ni T \rightarrow \text{Expr } \Gamma_2 \ (T \dots^S \eta)$$


```

393 data Value : Expr  $\Gamma$   $T \rightarrow$  Set where
394    $\lambda x : (e : \text{Expr } (\Gamma \triangleright T_1) T_2) \rightarrow \text{Value } (\lambda x \ e)$ 
395    $\Lambda \alpha : \text{Value } e \rightarrow \text{Value } (\Lambda \alpha \ e)$ 
396
397 data Progress : Expr  $\Gamma$   $T \rightarrow$  Set where
398   done : ( $v : \text{Value } e$ )  $\rightarrow$  Progress  $e$ 
399   step : ( $e \rightarrow e' : e \rightarrow e'$ )  $\rightarrow$  Progress  $e$ 
400
401 NoVar : Ctx  $n \rightarrow$  Set
402 NoVar  $\emptyset = \top$ 
403 NoVar ( $\Gamma \triangleright x$ ) =  $\perp$ 
404 NoVar ( $\Gamma \triangleright *$ ) = NoVar  $\Gamma$ 
405 noVar : NoVar  $\Gamma \rightarrow \neg (\Gamma \ni T)$ 
406 noVar  $nv(\text{suc}^* x) = \text{noVar } nv \ x$ 
407
408 progress : NoVar  $\Gamma \rightarrow (e : \text{Expr } \Gamma \ T) \rightarrow \text{Progress } e$ 
409 progress  $nv('x) = \perp\text{-elim } (\text{noVar } nv \ x)$ 
410 progress  $nv(\lambda x \ e) = \text{done } (\lambda x \ e)$ 
411 progress  $nv(e \cdot e_1)$ 
412   with progress  $nv \ e$ 
413   ... | done  $(\lambda x \ e_2) = \text{step } \beta\text{-}\lambda$ 
414   ... | step  $e \rightarrow e' = \text{step } (\xi\text{-} e \rightarrow e')$ 
415 progress  $nv(\Lambda \alpha \ e)$ 
416   with progress  $nv \ e$ 
417   ... | done  $v = \text{done } (\Lambda \alpha \ v)$ 
418   ... | step  $e \rightarrow e' = \text{step } (\xi\text{-}\Lambda \ e \rightarrow e')$ 
419 progress  $nv(e \cdot * T')$ 
420   with progress  $nv \ e$ 
421   ... | done  $(\Lambda \alpha \ v) = \text{step } \beta\text{-}\Lambda$ 
422   ... | step  $e \rightarrow e' = \text{step } (\xi\text{-}* \ e \rightarrow e')$ 

```

Fig. 5. Progress for System F

The identity substitution for expressions is straightforward to define. But this simplicity is due to the installed rewriting of type substitutions, which avoids an explicit application of a transport lemma using `comp-idr`.

$\text{Id}^S : \text{id}^S \mid \Gamma \Rightarrow^S \Gamma$

$\text{Id}^S _ = _$

Extending a substitution is standard.

$_ \cdot _ \cdot^S : \forall \eta \rightarrow \text{Expr } \Gamma_2 (T \dots^S \eta) \rightarrow \eta \mid \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \eta \mid (\Gamma_1 \triangleright T) \Rightarrow^S \Gamma_2$

$(_ \mid e \cdot^S \sigma) _ \text{zero} = e$

$(_ \mid e \cdot^S \sigma) _ (\text{suc } x) = \sigma _ x$

Due to the structure of contexts, we need two lifting lemmas, one for value bindings and another for type bindings. The former requires a transport lemma, which is applied by rewriting.

$_ \mid \sigma \uparrow^S : \forall \eta \rightarrow \eta \mid \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \forall T \rightarrow \eta \mid (\Gamma_1 \triangleright T) \Rightarrow^S (\Gamma_2 \triangleright (T \dots^S \eta))$

$\eta \mid \sigma \uparrow^S T = \eta \mid (' \text{zero}) \cdot^S \lambda _ x \rightarrow \text{id}^R \mid (\sigma _ x) \dots^R (\text{Wk}^R _)$

Lifting a substitution over a type binding is straightforward.

$_ \mid \sigma \uparrow^{S*} : \forall \eta \rightarrow \eta \mid \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow (\eta \uparrow^S) \mid (\Gamma_1 \triangleright^*) \Rightarrow^S (\Gamma_2 \triangleright^*)$

$(\eta \mid \sigma \uparrow^{S*}) _ (\text{suc}^* x) = _ \mid (\sigma _ x) \dots^R \text{wk}^{R*}$

These operations are sufficient to define the application of a substitution to an expression.

$_ \mid \dots^S : (\eta : n_1 \rightarrow^S n_2) \rightarrow \text{Expr } \Gamma_1 \ T \rightarrow \eta \mid \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \text{Expr } \Gamma_2 (T \dots^S \eta)$

$\eta \mid ('x) \dots^S \sigma = \sigma _ x$

$\eta \mid (\lambda x \ e) \dots^S \sigma = \lambda x (\eta \mid e \dots^S (\eta \mid \sigma \uparrow^S _))$

$\eta \mid (\Lambda \alpha \ e) \dots^S \sigma = \Lambda \alpha ((\eta \uparrow^S) \mid e \dots^S (\eta \mid \sigma \uparrow^{S*}))$

$\eta \mid (e \cdot e_1) \dots^S \sigma = (\eta \mid e \dots^S \sigma) \cdot (\eta \mid e_1 \dots^S \sigma)$

$\eta \mid (e \cdot * T') \dots^S \sigma = (\eta \mid e \dots^S \sigma) \cdot * (T' \dots^S \eta)$

2.4 Semantics

We are now in position to define a small-step operational semantics. The definition of single substitution, needed for beta reduction, is analogous to the one on types: we build the substitution by extending the identity substitution with the argument. In addition, a type substitution into expression is needed for beta reduction of type applications. This substitution is also implemented by an expression substitution indexed by a type substitution that performs the actual change.

$$\begin{aligned} _[_] : \text{Expr } (\Gamma \triangleright T) \rightarrow \text{Expr } \Gamma \quad T' \rightarrow \text{Expr } \Gamma \quad T \\ e[_] = \text{id}^S \mid e \dots^S (\text{id}^S \mid e' \dots^S \text{id}^S) \\ _[*] : \text{Expr } (\Gamma \triangleright^* T) \rightarrow (T : \text{Type } n) \rightarrow \text{Expr } \Gamma \quad (T[_]^*) \\ e[*] = (T \dots^S \text{id}^S) \mid e \dots^S (\text{id}^S \mid T \dots^S \text{id}^S) \end{aligned}$$

With substitution in place, the definition of call-by-name small-step reduction is straightforward. As usual, such a definition of reduction for intrinsically-typed syntax implies subject reduction by construction.

$$\begin{aligned} \text{data } \longrightarrow : \text{Expr } \Gamma \quad T \rightarrow \text{Expr } \Gamma \quad T \rightarrow \text{Set where} \\ \beta\text{-}\lambda : (\lambda x \ e_1 \cdot e_2) \longrightarrow (e_1[_] \ e_2) \\ \beta\text{-}\Lambda : (\Lambda \alpha \ e \cdot^* T) \longrightarrow (e[_] \ T^*) \\ \xi\text{-}\cdot : e_1 \longrightarrow e_1' \rightarrow (e_1 \cdot e_2) \longrightarrow (e_1' \cdot e_2) \\ \xi\text{-}\cdot^* : e \longrightarrow e' \rightarrow (e \cdot^* T) \longrightarrow (e' \cdot^* T) \\ \xi\text{-}\Lambda : e \longrightarrow e' \rightarrow (\Lambda \alpha \ e) \longrightarrow (\Lambda \alpha \ e') \end{aligned}$$

To obtain type safety, it remains to prove progress. ?? presents the complete proof of progress for call-by-name System F. A value is either a lambda or a big lambda where the body is already a value. An expression fulfills progress if it is either a value (**done**) or it can make a step (**step**). Due to the definition of value for big lambdas, we establish progress for type contexts that may contain type bindings, but no value bindings. These contexts are characterized by function **NoVar**. The actual proof of progress is by induction on the structure of expressions. The overall structure of this proof closely follows the corresponding proof by [?]. Their proof is more complicated because their semantics is call-by-value and their language supports isorecursive types.

2.5 Laws for Renamings and Substitutions

Proving type safety for System F *requires* exercising the substitution theory at the *type level*. Without installing its laws as definitional equalities, many definitions become more cumbersome because transport lemmas have to be inserted to equalize the types of (Agda) expressions.

While proving type safety does *not* require exercising same theory at the *expression level*, it is still good practice to verify functorial and monadic properties of expression substitutions. Moreover, if we were to move on to constructing and proving properties of logical relations, then these properties would be needed [?].

As an example, we examine the composition of two expression substitutions. Its definition involves two type substitutions η_1 and η_2 that index the corresponding expression substitutions σ_1 and σ_2 . The composed expression substitution is indexed by the composition of η_1 and η_2 . Type checking this definition already exercises the laws for composition of type substitutions.

$$\begin{aligned} _ \circ _ : \forall \eta_1 \eta_2 \rightarrow \eta_1 \mid \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \eta_2 \mid \Gamma_2 \Rightarrow^S \Gamma_3 \rightarrow (\eta_1 \circ \eta_2) \mid \Gamma_1 \Rightarrow^S \Gamma_3 \\ (_ \circ _ \mid \sigma_1 \circ \sigma_2) _ x = _ \mid (\sigma_1 _ x) \dots^S \sigma_2 \end{aligned}$$

Here is the statement of functoriality of expression substitution: applying two substitutions one after the other to an expression is the same as applying their composition. This statement is a very

prominent example of transfer hell: the types of the left and right side of the equality are different and would require a transfer without our rewriting machinery in place.

$\text{Compose}^{SS} : \forall (e : \text{Expr } \Gamma_1 T) (\eta_1 : n_1 \rightarrow^S n_2) (\eta_2 : n_2 \rightarrow^S n_3) (\sigma_1 : \eta_1 \mid \Gamma_1 \Rightarrow^S \Gamma_2) (\sigma_2 : \eta_2 \mid \Gamma_2 \Rightarrow^S \Gamma_3) \rightarrow$
 $\eta_2 \mid (\eta_1 \mid e \dots^S \sigma_1) \dots^S \sigma_2 \equiv (\eta_1 \circ \eta_2) \mid e \dots^S (\eta_1, \eta_2 \mid \sigma_1 \circ \sigma_2)$

We show the proof of this property just do demonstrate that type adjustments for type substitutions can be ignored and we can concentrate on the gist of the proof, rather than being overwhelmed by transfers. The remaining fly in the ointment is the explicit passing of the type substitutions η_1 and η_2 . We chose to make them explicit; while Agda can infer them in many places, there are equally many places where inference is not possible.

$\text{Compose}^{SS} (\text{' } x) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 = \text{refl}$

$\text{Compose}^{SS} (\lambda x e) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 = \text{cong } \lambda x (\text{begin}$
 $\quad \quad \quad \equiv \langle \text{Compose}^{SS} e \eta_1 \eta_2 (\eta_1 \mid \sigma_1 \uparrow^S _) (\eta_2 \mid \sigma_2 \uparrow^S _) \rangle$
 $\quad \quad \quad \equiv \langle \text{cong } ((\eta_1 \circ \eta_2) \mid e \dots^S _) (\text{Lift-Dist-Comp}^{SS} \sigma_1 \sigma_2) \rangle$
 $\quad \quad \quad \equiv \square)$

$\text{Compose}^{SS} (\Lambda \alpha e) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 = \text{cong } \Lambda \alpha (\text{begin}$
 $\quad \quad \quad \equiv \langle \text{Compose}^{SS} e (\eta_1 \uparrow^S) (\eta_2 \uparrow^S) (\eta_1 \mid \sigma_1 \uparrow^{S*}) (\eta_2 \mid \sigma_2 \uparrow^{S*}) \rangle$
 $\quad \quad \quad \equiv \langle \text{cong } (((\eta_1 \circ \eta_2) \uparrow^S) \mid e \dots^S _) (\text{lift}^*\text{-dist-Comp}^{SS} \eta_1 \eta_2 \sigma_1 \sigma_2) \rangle$
 $\quad \quad \quad \equiv \square)$

$\text{Compose}^{SS} (e_1 \cdot e_2) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 = \text{cong}_2 _ _ (\text{Compose}^{SS} e_1 \eta_1 \eta_2 \sigma_1 \sigma_2) (\text{Compose}^{SS} e_2 \eta_1 \eta_2 \sigma_1 \sigma_2)$

$\text{Compose}^{SS} (e \cdot^* T) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 = \text{cong } (_ \cdot^* (T \dots^S (\eta_1 \circ \eta_2))) (\text{Compose}^{SS} e \eta_1 \eta_2 \sigma_1 \sigma_2)$

This proof relies on further properties, in particular $\text{Lift-Dist-Comp}^{SS}$ (lifting a composition of substitutions is equal to composing their liftings) and $\text{lift}^*\text{-dist-Comp}^{SS}$ (its counterpart for lifting over type bindings), where the statement and proof would require explicit transfers. Essentially, these properties form a “food chain” that stops at the composition properties of renamings with renamings and liftings, all with similar issues. The supplemental material gives the full story.

2.6 Comparison to Proof in Transfer Hell

We compare this proof to the one without the additional definitional equalities for type substitutions, provided in the supplement for the aforementioned logical relation construction [?]. Here we only state the *type* of the proof and the induction case for application, which is among the short ones.

$(e : \text{Expr } \Delta_1 \Gamma_1 T) (\sigma_1 : \text{ESub } \sigma_1^* \Gamma_1 \Gamma_2) \rightarrow (\sigma_2 : \text{ESub } \sigma_2^* \Gamma_2 \Gamma_3) \rightarrow$
 $\text{let } \text{sub} = \text{subst } (\text{Expr } \Delta_3 \Gamma_3) (\text{fusion-Tsub-Tsub } T \sigma_1^* \sigma_2^*) \text{ in}$
 $\text{sub } (\text{ESub } \sigma_2^* \sigma_2 (\text{ESub } \sigma_1^* \sigma_1 e)) \equiv \text{ESub } (\sigma_1^* \circ_{SS} \sigma_2^*) (\sigma_1 \gg^{SS} \sigma_2) e$

The mapping from their to our syntax (in order) is

- $\Delta \mapsto n$
- $\text{ESub} \mapsto _ \mid \dots^S _$
- $\sigma^* \mapsto \eta$
- $\text{fusion-Tsub-Tsub} \mapsto \text{compose}^{SS}$
- $_ \circ_{SS} _ \mapsto _ \circ _$
- $_ \gg^{SS} _ \mapsto _ \mid _ \circ _$

The authors use heterogeneous equality to proof the lemma, thereby avoiding some of the troubles arising from transfer hell.

R.begin

$\text{ESub } \sigma_2^* \sigma_2 (\text{ESub } \sigma_1^* \sigma_1 (e_1 \cdot e_2))$

```

540 R.≡⟨ refl ⟩
541   Esub σ2* σ2 (Esub σ1* σ1 e1) · Esub σ2* σ2 (Esub σ1* σ1 e2)
542 R.≡⟨ Hcong4 {C = λ ■1 ■2 → Expr Δ3 Γ3 (■2 ⇒ ■1)} {D = λ _ ■2 _ → Expr Δ3 Γ3 ■2} (λ _ _ → _ · _)
543   (H.≡-to-≡ (fusion-Tsub-Tsub T σ1* σ2*)) (H.≡-to-≡ (fusion-Tsub-Tsub T1 σ1* σ2*))
544   (fusion-Esub-Esub' e1 σ1 σ2) (fusion-Esub-Esub' e2 σ1 σ2) ⟩
545   Esub (σ1* ○ss σ2*) (σ1 »SS σ2) e1 · Esub (σ1* ○ss σ2*) (σ1 »SS σ2) e2
546 R.≡⟨ refl ⟩
547   Esub (σ1* ○ss σ2*) (σ1 »SS σ2) (e1 · e2)
548 R.□

```

The full proof can be found in Appendix ??

2.7 Examples

As an example of System F expressions, we present some definitions for Church numerals. We start with their type. For readability, we define aliases for type variables like ' α ' for ' Z ', ' β ' for ' $(S Z)$ ', and analogously for expression variables and binders.

```

556 Nc : Type 0 *
557 Nc = ∀α ((α ⇒ α) ⇒ (α ⇒ α))
558 We give the definitions for some constants and the successor operation.
559 zeroc : Expr 0 Nc
560 zeroc = Λα (λg (λx 'x))
561 onec : Expr 0 Nc
562 onec = Λα (λg (λx ('g · 'x)))
563 succc : Expr 0 (Nc ⇒ Nc)
564 succc = λx (Λα (λg (λx ('g · ((((' (S (S (S* Z)))) • 'α) · 'g) · 'x)))))
565 twoc : Expr 0 Nc
566 twoc = succc · (succc · zeroc)

```

3 System F ω

System F ω extends System F with higher-kinded types, that is, abstraction over type functions. This calculus is the basis of Haskell's type system where type functions are limited to partial application of type constructors. Abstraction of higher-kinded types is key to defining type classes for functors, monads, etc. An extension of F ω with isorecursive types is the basis for Plutus, the language of the Cardano blockchain [?].

3.1 Definitions

We only give a glimpse of the definitions of the syntax of types and expressions, which are taken from [?] and hence generalize the definitions for System F from ??. We do not reiterate the development of renamings and substitutions, as it transfers mutatis mutandis: the typings get more involved, but the equational laws remain exactly the same.

For System F ω , type variables have more structure as in System F, as there are arrow kinds in addition to plain types. We use I, J, K to range over types.

```

585 data Kind : Set where
586   * : Kind
587   _⇒_ : Kind → Kind → Kind

```

Consequently, kind contexts are not mere numbers, but lists of kinds that grow to the right. Kind contexts are ranged over by Φ, Ψ, Θ .

```
data Ctx* : Set where
  ∅ : Ctx*
  _▷* _ : Ctx* → Kind → Ctx*
```

Type variables have the same structure as plain variables in the simply-typed lambda calculus (see ??).

```
data _▷* _ : Ctx* → Kind → Set where
  Z : (Φ ▷* K) ▷* K
  S : Φ ▷* K → (Φ ▷* J) ▷* K
```

The definition of the type syntax also mimicks the expressions of the simply-typed lambda calculus, but with an important twist, which deviates from the prior work.

```
data Type Φ : Kind → Set where
  ' _ : Φ ▷* K → Type Φ K
  λα : Type (Φ ▷* J) K → Type Φ (J ⇒ K)
  - _$ _ : Type Φ (J ⇒ K) → Type Φ J → Type Φ K
  ∀α : Type (Φ ▷* J) * → Type Φ *
  _⇒ _ : Type Φ * → Type Φ * → Type Φ *
```

postulate

```
_$_ : Type Φ (J ⇒ K) → Type Φ J → Type Φ K
```

As System $F\omega$ enjoys full beta reduction at the type level, we do not define application (with symbol $\$$) as a data constructor. Instead, we introduce it as a symbol and use rewriting to extend functions on the type syntax to include application.

As an example, we show the definition of applying a renaming to an expression. We elide the preceding definitions of lifting, extension, and the type substitution machinery because they are identical to the ones for System F in ??, save for the more elaborate type signatures.

```
_...R _ : Type Φ K → Φ →R Ψ → Type Ψ K
(' x) ...R ρ = ' ρ x
(λα T) ...R ρ = λα (T ...R ↑R ρ)
- (T1 $ T2) ...R ρ = (T1 ...R ρ) $ (T2 ...R ρ)
(∀α T) ...R ρ = ∀α (T ...R ↑R ρ)
(T1 ⇒ T2) ...R ρ = (T1 ...R ρ) ⇒ (T2 ...R ρ)
```

postulate

```
traverseR-$ : (T1 $ T2) ...R ρ ≡ (T1 ...R ρ) $ (T2 ...R ρ)
```

On top of the rewriting theory for type substitutions, we install type-level beta reduction as a rewrite rule, using single type substitution on the right side:

postulate

```
β≡* : (λα T1) $ T2 ≡ T1 [ T2 ]*
```

```
{-# REWRITE β≡* #-}
```

The combined theory is still confluent and terminating [? ?].

Surprisingly, type-level beta reduction is not required to define the syntax of expressions, expression renamings and substitutions, defining the operational semantics, as well as the proof of type safety. Thanks to definitional beta conversion at the type level, the expression syntax does

not require an explicit type conversion rule as it was the case in prior work [?]. Hence, the proof presented in ?? for System F carries over, mutatis mutandis (i.e., adapting the type signatures).

In the next section, we develop some examples that genuinely rely on type conversion.

3.2 Examples

?] consider several examples to demonstrate their encoding of System $F\omega\mu$. Their first group of examples covers Church numerals in System F. These are covered in ??. Their second group of example covers recursive types. We have no counterpart for them. Unfortunately, they do not provide examples that exercise the full power of System $F\omega$, namely higher-kinded types! We fill this gap in the following.

3.2.1 Church Numerals. First, we encode (simply-typed) Church numerals *in the type language*. As beta reduction on the type level is definitional in our development, normalization of Church numerals is performed by Agda's type checker.

The kind of type level Church numerals is as expected.

$\mathbb{N}^k$: Kind

$\mathbb{N}^k = (* \Rightarrow *) \Rightarrow (* \Rightarrow *)$

We define zero and successor for Church numerals in the usual way. For readability, we define aliases like $\lambda\beta$ and so on for the type level lambda $\lambda\alpha$. Analogously, we define aliases for type variables like α for 'Z , β for '(S Z) , and so on.

zero^k : Type $\emptyset \mathbb{N}^k$

$\text{zero}^k = \lambda\beta (\lambda\alpha \text{'}\alpha)$

succ^k : Type $\emptyset (\mathbb{N}^k \Rightarrow \mathbb{N}^k)$

$\text{succ}^k = \lambda\gamma (\lambda\beta (\lambda\alpha (\text{'}\beta \$ ((\text{'}\gamma \$ \text{'}\beta) \$ \text{'}\alpha))))$

We construct the Church numeral for 2 by applying the successor twice to zero. The proof that this term is equal to its normal form is definitional.

two^k : Type $\emptyset \mathbb{N}^k$

$\text{two}^k = \text{succ}^k \$ (\text{succ}^k \$ \text{zero}^k)$

$_ : \text{two}^k \equiv \lambda\beta (\lambda\alpha (\text{'}\beta \$ (\text{'}\beta \$ \text{'}\alpha)))$

$_ = \text{refl}$

Finally, we define addition and construct 2 by adding 1 + 1. The corresponding equality proof puts a significant load on the type checker and takes more than a minute to complete on the authors' machine.

one^k : Type $\emptyset \mathbb{N}^k$

$\text{one}^k = \lambda\beta (\lambda\alpha (\text{'}\beta \$ \text{'}\alpha))$

add^k : Type $\emptyset (\mathbb{N}^k \Rightarrow (\mathbb{N}^k \Rightarrow \mathbb{N}^k))$

$\text{add}^k = \lambda\alpha (\lambda\alpha (\lambda\alpha (\lambda\alpha (((\text{'} S (S (S Z))) \$ (\text{'} S Z)) \$ (((\text{'} S (S Z)) \$ (\text{'} S Z)) \$ (\text{'} Z))))))$

$_ : \text{two}^k \equiv (\text{add}^k \$ \text{one}^k) \$ \text{one}^k$

$_ = \text{refl}$

3.2.2 Arrows. The final example is inspired by the work on arrows [?]. This work uses type classes to abstract over different instantiations of the function arrow. For clarity, we start with an Agda version of the function that we wish to implement in System $F\omega$.

The function `abst` is a glorified identity function. It abstracts over a binary type constructor κ , two types β and α , then it takes an abstract application function `app` and applies it to an abstract function `f` and a suitable argument `x`. Function `inst` instantiates this function. It acts like an eliminator for `abst`. The function `use` puts both together.

```

691 ty : Set1
692 ty = ( $\kappa : \text{Set} \rightarrow \text{Set} \rightarrow \text{Set}$ ) ( $\beta \alpha : \text{Set}$ ) ( $\text{app} : \kappa \beta \alpha \rightarrow \beta \rightarrow \alpha$ ) ( $f : \kappa \beta \alpha$ ) ( $x : \beta$ )  $\rightarrow \alpha$ 
693
694 abst : ty
695 abst =  $\lambda \kappa \beta \alpha \rightarrow \lambda \text{app } f x \rightarrow \text{app } f x$ 
696
697 inst :  $\forall \alpha \rightarrow \text{ty} \rightarrow \alpha \rightarrow \alpha$ 
698 inst =  $\lambda \alpha g x \rightarrow g (\lambda \beta \alpha \rightarrow (\beta \rightarrow \alpha)) \alpha \alpha (\lambda f x \rightarrow f x) (\lambda z \rightarrow z) x$ 
699
700 use :  $\forall \alpha \rightarrow \alpha \rightarrow \alpha$ 
701 use =  $\lambda \alpha x \rightarrow \text{inst } \alpha \text{ abst } x$ 

```

We start by defining the type in our encoding.

```

702 ty-enc : Type  $\Phi^*$ 
703 ty-enc =  $\forall \kappa (\forall \beta (\forall \alpha (((\kappa \$ \beta) \$ \alpha) \Rightarrow (\beta \Rightarrow \alpha))$ 
704            $\Rightarrow (((\kappa \$ \beta) \$ \alpha) \Rightarrow (\beta \Rightarrow \alpha))))$ 
705

```

Next, we encode the defining expression equivalent to `abst`.

```

706
707 abst-enc : Expr  $\Gamma$  ty-enc
708 abst-enc =  $\Delta \kappa (\Delta \beta (\Delta \alpha (\lambda f (\lambda y (\lambda x ((f \cdot y) \cdot x))))$ 
709
710 To use this function, we encode functions inst and use.
711
712 inst-enc : Expr  $\Gamma$  ( $\forall \alpha (\text{ty-enc} \Rightarrow (\alpha \Rightarrow \alpha))$ 
713 inst-enc =  $\Delta \alpha (\lambda g (\lambda x ((((((g \cdot (\lambda \beta (\lambda \alpha (\beta \Rightarrow \alpha))) \cdot \alpha) \cdot \alpha)$ 
714                                $\cdot (\lambda y (\lambda x (y \cdot x)))$ 
715                                $\cdot (\lambda x \text{'x'})$ 
716                                $\cdot \text{'x'})$ 
717

```

```

717 use-enc : Expr  $\emptyset$  ( $\forall \alpha (\alpha \Rightarrow \alpha)$ 
718 use-enc =  $\Delta \alpha (\lambda x (((\text{inst-enc} \cdot \alpha) \cdot \text{abst-enc}) \cdot \text{'x'})$ 

```

The definitions of the encodings require beta conversion at the type level. These definitions are not accepted without the rewriting of beta reduction $\beta \equiv^*$.

4 Conclusion

Intrinsically typed syntax promises elegant mechanizations by making well-typed programs representable only as well-typed data, but for polymorphic calculi this promise is often undermined by pervasive transport reasoning. Our experience with System F and System F ω shows that the difficulty does not stem from binding structure or indexing itself, but from a mismatch between substitution and definitional equality: substitutions that are extensionally equal fail to compute to a canonical form. By internalizing the equational laws of the σ -calculus as definitional equality, substitutions become canonical and routine transports disappear from proofs. The resulting developments recover the simplicity seen in the simply-typed case while scaling to higher-order polymorphism and intrinsic type conversion. More broadly, this pearl suggests a practical guideline for mechanizing type systems in dependently typed proof assistants: when intrinsically typed encodings become proof-heavy, a promising strategy is often to strengthen definitional equality rather than introduce additional lemmas or automation.

A Proof for Compose Monad Law in Transfer Hell

```

736 fusion-Esub-Esub {Δ1} {Δ2} {Δ3} {ℓ} {σ*} {ρ*} {Γ1} {Γ2} {Γ3} {T} e σ ρ =
737 let F1 = (Expr _ Γ3) ; E1 = (fusion-Tsub-Tsub T σ* ρ*) ; sub1 = subst F1 E1 in
738 H.≡-to-≡ (
739   R.begin
740     sub1 (Esub ρ* ρ (Esub σ* σ e))
741   R.≡⟨ H.≡-subst-removable F1 E1 _ ⟩
742     Esub ρ* ρ (Esub σ* σ e)
743   R.≡⟨ fusion-Esub-Esub' e σ ρ ⟩
744     Esub (σ* ∘ss ρ*) (σ »SS ρ) e
745   R.□
746 )
747 fusion-Esub-Esub' :
748   {σ1* : TSub Δ1 Δ2} {σ2* : TSub Δ2 Δ3}
749   → {Γ1 : TEnv Δ1} {Γ2 : TEnv Δ2} {Γ3 : TEnv Δ3}
750   → {T : Type Δ1 ℓ}
751   → (e : Expr Δ1 Γ1 T)
752   → (σ1 : Esub σ1* Γ1 Γ2) (σ2 : Esub σ2* Γ2 Γ3)
753   → Esub σ2* σ2 (Esub σ1* σ1 e) ≡ Esub (σ1* ∘ss σ2*) (σ1 »SS σ2) e
754 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {zero} {σ1*} {σ2*} {Γ1} {Γ2} {Γ3} {'N} (# n) σ1 σ2 =
755 refl
756 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {zero} {σ2*} {σ1*} {Γ1} {Γ2} {Γ3} {'N} ('suc e) σ2 σ1 =
757   R.begin
758     Esub σ1* σ1 (Esub σ2* σ2 ('suc e))
759   R.≡⟨ refl ⟩
760     'suc (Esub σ1* σ1 (Esub σ2* σ2 e))
761   R.≡⟨ H.cong 'suc (fusion-Esub-Esub' e σ2 σ1) ⟩
762     'suc (Esub (σ2* ∘ss σ1*) (σ2 »SS σ1) e)
763   R.≡⟨ refl ⟩
764     Esub (σ2* ∘ss σ1*) (σ2 »SS σ1) ('suc e)
765   R.□
766 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {ℓ} {σ1*} {σ2*} {Γ1} {Γ2} {Γ3} {T} ('x) σ1 σ2 =
767 let F1 = (Expr Δ3 Γ3) ; E1 = (fusion-Tsub-Tsub T σ1* σ2*) ; sub1 = subst F1 E1 in
768   R.begin
769     Esub σ2* σ2 (Esub σ1* σ1 ('x))
770   R.≡⟨ refl ⟩
771     Esub σ2* σ2 (σ1 l T x)
772   R.≡⟨ H.sym (H.≡-subst-removable F1 E1 _) ⟩
773     sub1 (Esub σ2* σ2 (σ1 l T x))
774   R.≡⟨ refl ⟩
775     Esub (σ1* ∘ss σ2*) (σ1 »SS σ2) ('x)
776   R.□
777 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {⊥} {σ1*} {σ2*} {Γ1} {Γ2} {Γ3} {T1 ⇒ T2} (λ e) σ1 σ2 =
778   R.begin
779     λ Esub σ2* (Elifts σ2* σ2) (Esub σ1* (Elifts σ1* σ1) e)
780   R.≡⟨ Hcong {C = λ ■1 ■2 → Expr Δ3 (■1 < Γ3) ■2} (λ _ _ → λ ■)
781     (H.≡-to-≡ (fusion-Tsub-Tsub T1 σ1* σ2*))
782     (H.≡-to-≡ (fusion-Tsub-Tsub T2 σ1* σ2*))
783     (fusion-Esub-Esub-lift e σ1 σ2) ⟩
784     λ (Esub (σ1* ∘ss σ2*) (Elifts (σ1* ∘ss σ2*) (σ1 »SS σ2)) e)
785   R.≡⟨ refl ⟩
786     Esub (σ1* ∘ss σ2*) (σ1 »SS σ2) (λ e)
787   R.□

```



```

785   R.□
786 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {l} {σ1*} {σ2*} {Γ1} {Γ2} {Γ3} {T} (λ _ {T = T1} {T' = T2} e1 e2) σ1 σ2 =
787   R.begin
788     Esub σ2* σ2 (Esub σ1* σ1 (e1 · e2))
789   R.≡( refl )
790     Esub σ2* σ2 (Esub σ1* σ1 e1) · Esub σ2* σ2 (Esub σ1* σ1 e2)
791   R.≡( Hcong4 {C = λ ■ ■2 → Expr Δ3 Γ3 (■2 ⇒ ■1)} {D = λ _ ■2 _ → Expr Δ3 Γ3 ■2} (λ _ _ → _ · _)
792     (H.≡-to-≡ (fusion-Tsub-Tsub T σ1* σ2*)) (H.≡-to-≡ (fusion-Tsub-Tsub T1 σ1* σ2*))
793     (fusion-Esub-Esub' e1 σ1 σ2) (fusion-Esub-Esub' e2 σ1 σ2) )
794     Esub (σ1* oss σ2*) (σ1 »SS σ2) e1 · Esub (σ1* oss σ2*) (σ1 »SS σ2) e2
795   R.≡( refl )
796     Esub (σ1* oss σ2*) (σ1 »SS σ2) (e1 · e2)
797   R.□
798 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {λ} {σ1*} {σ2*} {Γ1} {Γ2} {Γ3} {‘∀α l, T} (λ l ⇒ e) σ1 σ2 =
799   R.begin
800     Esub σ2* σ2 (Esub σ1* σ1 (λ l ⇒ e))
801   R.≡( refl )
802     λ l ⇒ Esub (Tlifts σ2* l) (Elifts-l σ2* σ2) (Esub (Tlifts σ1* l) (Elifts-l σ1* σ1) e)
803   R.≡( H.cong2 {B = Expr (l :: Δ3) (l < * Γ3)} (λ _ → λ l ⇒ _)
804     (H.≡-to-≡ (fusion-Tsub-Tsub T (Tlifts σ1* _) (Tlifts σ2* _)))
805     (fusion-Esub-Esub' e (Elifts-l σ1* σ1) (Elifts-l σ2* σ2)) )
806     λ l ⇒ Esub (Tlifts σ1* l oss Tlifts σ2* l) (Elifts-l σ1* σ1 »SS Elifts-l σ2* σ2) e
807   R.≡( H.cong2 {B = Expr (l :: Δ3) (l < * Γ3)} (λ _ → λ l ⇒ _)
808     (H.≡-to-≡ (cong (λ σ1 → Tsub σ1 T) (sym (sub↑-dist-oss _ σ1* σ2*))))
809     (fusion-Esub-Esub-lift-l e σ1 σ2) )
810     λ l ⇒ Esub (Tlifts (σ1* oss σ2*) l) (Elifts-l (σ1* oss σ2*) (σ1 »SS σ2)) e
811   R.≡( refl )
812     Esub (σ1* oss σ2*) (σ1 »SS σ2) (λ l ⇒ e)
813   R.□
814 fusion-Esub-Esub' {Δ1} {Δ2} {Δ3} {l} {σ1*} {σ2*} {Γ1} {Γ2} {Γ3} {λ} (λ _ {T = T} e T') σ1 σ2 =
815   let
816     F2 = Expr Δ2 Γ2 ; E2 = sym (swap-Tsub-[] σ1* T T') ; sub2 = subst F2 E2
817     F3 = Expr Δ3 Γ3 ; E3 = sym (swap-Tsub-[] (σ1* oss σ2*) T T') ; sub3 = subst F3 E3
818     F5 = Expr Δ3 Γ3 ; E5 = sym (swap-Tsub-[] σ2* (Tsub (Tlifts σ1* _) T) (Tsub σ1* T')) ; sub5 = subst F5 E5
819   in
820   R.begin
821     Esub σ2* σ2 (Esub σ1* σ1 (e · T'))
822   R.≡( refl )
823     Esub σ2* σ2 (sub2 (Esub σ1* σ1 e · Tsub σ1* T'))
824   R.≡( H.cong2 {B = Expr Δ2 Γ2} (λ _ ■ → Esub σ2* σ2 ■) (H.≡-to-≡ (sym E2))
825     (H.≡-subst-removable F2 E2 _) )
826     Esub σ2* σ2 (Esub σ1* σ1 e · Tsub σ1* T')
827   R.≡( refl )
828     sub5 (Esub σ2* σ2 (Esub σ1* σ1 e) · Tsub σ2* (Tsub σ1* T'))
829   R.≡( H.≡-subst-removable F5 E5 _ )
830     Esub σ2* σ2 (Esub σ1* σ1 e) · Tsub σ2* (Tsub σ1* T')
831   R.≡( Hcong3 {B = λ ■ → Expr Δ3 Γ3 (‘∀α _ , ■)} {C = λ _ _ → Type Δ3 _} (λ _ ■ ■2 → ■1 · ■2)
832     (H.≡-to-≡ (fusion-Tsub-Tsub-lift T σ1* σ2*)) (fusion-Esub-Esub' e σ1 σ2)
833     (H.≡-to-≡ (fusion-Tsub-Tsub T' σ1* σ2*)) )
834     Esub (σ1* oss σ2*) (σ1 »SS σ2) e · Tsub (σ1* oss σ2*) T'
835   R.≡( H.sym (H.≡-subst-removable F3 E3 _) )
836     sub3 (Esub (σ1* oss σ2*) (σ1 »SS σ2) e · (Tsub (σ1* oss σ2*) T'))

```

834 $R. \cong (\text{refl})$
835 $\text{Esub}(\sigma_1 * \circ_{ss} \sigma_2^*)(\sigma_1 \gg SS \sigma_2)(e \cdot T')$
836 $R. \square$
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882

Temporary page!

$\LaTeX$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\LaTeX$ now knows how many pages to expect for this document.